

Resolviendo parciales

Ejercicio 3 Responder V o F y justificar. Marque con un círculo la respuesta y justifique en hoja aparte.

- a) (7.5) Todos los lenguajes funcionales son fuertemente tipados. V **F**
b) (7.5) Los parámetros formales por resultado pueden usarse en ejecución tal como se reciben en el procedimiento o función V **F**

a: FALSO

- No todos los lenguajes que permiten el estilo de programación funcional son fuertemente tipados, tenemos el caso de Python que permite programación estilo funcional y Python es un lenguaje con tipado dinámico y débil.

b: FALSO

- Los parámetros por resultado (conocidos por modo OUT), no contienen un valor inicial cuando se le pasa al procedimiento o función. El procedimiento o función DEBE asignarle un valor antes de usarlo. Entonces NO se puede usar directamente al inicio de la función, su propósito es que se devuelva un valor al final de la función.

Ejercicio 5 a)(10) Dado el siguiente código en Python. Describa los posibles caminos de ejecución.

```
#!/usr/bin/env python
#calc.py
```

```
def imprimir_posicion(x):
    print(("Resultado"), a[x]/a[x]-4)
```

0, 1, 2, 3, 4, 5

```
#La función range devuelve los números desde 0 al límite enviado como parámetro menos 1
for x in range(6):
```

```
    try:
        a = [0, 1, 2, 3, 4]
        imprimir_posicion(x)
    except IndexError:
        print("Ocurrió un error en el índice")
```

Pág.

```
        print("Ocurrió un error en el índice")
    except ZeroDivisionError:
        print("Ocurrió una división por cero")
    else:
        print("Se pudo acceder correctamente")
    finally:
        print("Vuelve a probar")
```

Este programa tiene una funcion que se encarga de imprimir el resultado del valor del indice que recibe, dividido por el mismo valor -4

`'a[x] / (a[x]-4)'`

Se tiene un for de x in range (6): Arranca con X=0 y termina con X=5 (inclusive)

Caminos de ejecucion:

Iteración `x = 0` :

- Se inicializa `a = [0,1,2,3,4]`
- Se invoca a la funcion `imprimir_posicion(x)`
 - `a[x] = 0`
 - Imprime 'resultado ' de la operacion: 0 / -4
- No ocurre ninguna excepcion, entra al else
 - Imprime "Se pudo acceder correctamente"
 - continua con la siguiente iteracion
- Entra al bloque Finally
 - Imprime "Vuelve a probar"

Iteración `x = 1` :

- Se inicializa `a = [0,1,2,3,4]`
- Se invoca a la funcion `imprimir_posicion(x)`
 - `a[x] = 1`
 - Imprime 'resultado ' de la operacion: 1 / -3
- No ocurre ninguna excepcion, entra al else
 - Imprime "Se pudo acceder correctamente"
 - continua con la siguiente iteracion
- Entra al bloque Finally
 - Imprime "Vuelve a probar"

Iteración `x = 2` :

- Se inicializa `a = [0,1,2,3,4]`
- Se invoca a la funcion `imprimir_posicion(x)`
 - `a[x] = 2`
 - Imprime 'resultado ' de la operacion: 2 / -2
- No ocurre ninguna excepcion, entra al else
 - Imprime "Se pudo acceder correctamente"
 - continua con la siguiente iteracion

- Entra al bloque Finally
 - Imprime "Vuelve a probar"

Iteración `x = 3` :

- Se inicializa `a = [0,1,2,3,4]`
- Se invoca a la funcion `imprimir_posicion(x)`
 - `a[x] = 3`
 - Imprime 'resultado ' de la operacion: 3 / -1
- No ocurre ninguna excepcion, entra al else
 - Imprime "Se pudo acceder correctamente"
 - continua con la siguiente iteracion
- Entra al bloque Finally
 - Imprime "Vuelve a probar"

Iteración `x = 4` :

- Se inicializa `a = [0,1,2,3,4]`
- Se invoca a la funcion `imprimir_posicion(x)`
 - `a[x] = 4`
 - Imprime 'resultado ' de la operacion: 4 / 0
- Se produce una excepcion por dividir por cero
 - Imprime: "Ocurrió una division por cero"
- Entra al bloque Finally
 - Imprime "Vuelve a probar"

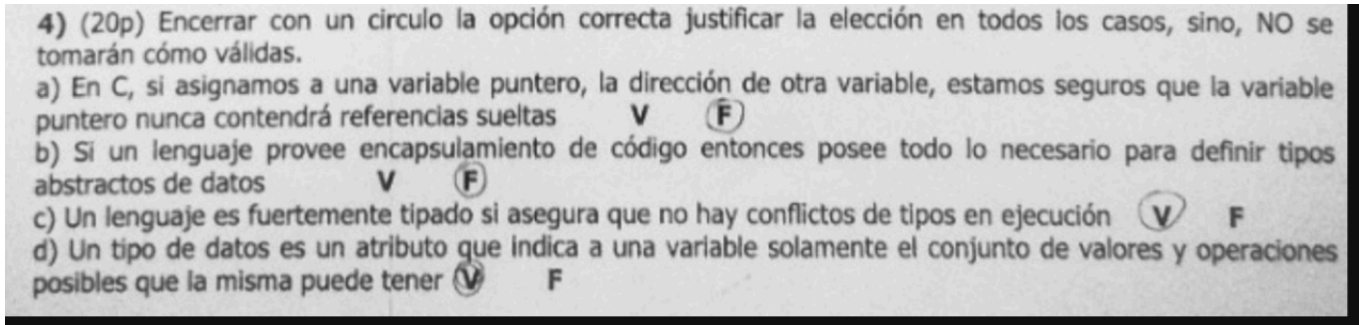
Iteración `x = 5` :

- Se inicializa `a = [0,1,2,3,4]`
- Se invoca a la funcion `imprimir_posicion(x)`
 - `a[x] = FUERA DE RANGO`
 - Al intentar acceder a un indice que esta fuera de rango, lanza una excepcion
- Se produce una excepcion `IndexError`
 - Imprime: "Ocurrió un error en el indice"
- Entra al bloque Finally
 - Imprime "Vuelve a probar"

Termina el for y termina el bloque de codigo que nos dieron.

Detalles importantes de este ejercicio:

- El bloque ELSE se ejecuta solamente si no ocurrió ninguna excepción.
- El bloque Finally se ejecuta SIEMPRE.



a: FALSO.

- Aunque asignemos una dirección válida a un puntero, ese puntero puede terminar apuntando a referencias sueltas, por ejemplo si la variable que apuntaba fue liberada con un free, o también si apuntaba a una variable local dentro de una función que ya terminó su ejecución.

b: FALSO.

- Para que un lenguaje provea todo lo necesario para definir tipo de datos abstractos (TAD), no solo es necesario proveer encapsulamiento, sino también debe proveer ocultamiento.
 - Encapsulamiento + ocultamiento => TADs

c: VERDADERO.

- Por definición de la teoría: Un lenguaje es fuertemente tipado si el sistema de tipos impone restricciones que aseguran que no se produzcan errores de tipo en ejecución.

d: VERDADERO.

- Es la definición que usamos en la cátedra, un tipo de dato es un atributo que indica solamente el conjunto de valores y operaciones permitidas para ese tipo de dato.

b)(10p). Indique cuáles son los tipos de datos identificados en el siguiente código C. Justifique en cada caso

```
#include <stdio.h>
#include <string.h>

struct Coordenadas{
    int x;
    int y;
};

union Persona
{
    char nombre[20];
    char apellido[20];
    long dni;
};

int sumaIndices(Coordenadas p)
{
    int result;
    result = p.x + p.y;
    return result;
}
```

Tipos de datos primitivos:

- int: Es un tipo de dato utilizado para almacenar números enteros (sin parte decimal).
- char: Es un tipo de dato utilizado para almacenar un único carácter, por ejemplo 'A', '1', '\$'. Generalmente los char ocupan 1 byte
- long: Es un tipo de dato similar a int, pero abarca un rango más grande de valores posibles de números enteros.

Tipos de datos compuestos (definidos por el usuario)

- El tipo de dato Coordenadas usa como constructor el producto cartesiano.
Es un producto cartesiano porque crea un tipo que tiene elementos con n tuplas ordenadas, donde cada campo, puede pertenecer a un tipo de dato diferente.
en este código: Se crea un registro al cual se accede con nombreVariable . nombreCampo.
- El tipo de dato Persona permite la manipulación de diferentes tipos en distintos momentos durante la ejecución.
- int sumaIndices (Coordenadas p): Utiliza el constructor de correspondencia finita. Establece que para cada valor del tipo de coordenadas, produce un único valor de salida de tipo int. Esta es justamente la definición de correspondencia finita.

Ejercicio 1 (30p)

Realice la pila de ejecución para el siguiente código:

- a) por cadena estática
- b) por cadena dinámica

Program Main;

Var x, y, z: integer;

a, b: array[1..6] of integer;

Procedure D(var x: integer; nombre y: integer);

var h: integer;

begin

x := f + 5;

y := y + 2;

h := x + 15;

end;

Function F: integer;

Var y: integer;

Begin

y := 1;

b(x) := b(x) + 1;

if (x < 6) **then**

x := x + 1;

a(y) := a(y) + b(x) + 3;

a(x) := a(x) + 2;

y := y + 3;

return b(y);

end

begin

x := 1; y := 1;

for z := 1 **to** 6 **do begin**

a(z) := abs(7-z);

b(z) := z * 2;

end;

D(a(z), b(abs(6-x+y)));

for z := 1 **to** 6 **do write** (a(z), b(z));

end.

Nota: La forma de evaluación de este lenguaje es de izquierda a derecha. Abs es una función que retorna el valor absoluto de la operación recibida.

Resolucion por cadena estatica.

Parcial 10/06/2023

CIENNA ESTADIA

1) MAIN - PR

LE

LD

X = 1, 2

Y = 1

Z = 1, 6

A[1] = 13

A[2] = 5, 7

A[3] = 4

A[4] = 3

A[5] = 2

A[6] = 1, 5

B[1] = 2, 3

B[2] = 4

B[3] = 6

B[4] = 8

B[5] = 10

B[6] = 12, 14

FUNCTION F

PROLOGIC D

VR

2) FUNCTION A - PR

LE

LD

X =

Y =

VR 8

3) FUNCTION F - PR

LE

LD

Y = 1, 4

VR

RETURN A(6)

D(a(z), b(ABS(6-x+y)))

For Z = 1 to 6
Z=1 a(1)=6
b(1)=2
Z=2 a(2)=5
b(2)=4
Z=3 a(3)=4
b(3)=6
Z=4 a(4)=3
b(4)=8
Z=5 a(5)=2
b(5)=10
Z=6 a(6)=1
b(6)=12

IMPRIME: (13, 3)
(7, 4)
(4, 6)
(3, 6)
(2, 10)
(13, 14)

$x = F + 5 \rightarrow x = 8 + 5$
 $y = y + 2 =$
 $b(ABS(6 - 2 + 1)) = b(ABS(6 - 2 + 1)) + 2$
 $b(6) = b(6) + 2 = 12 + 2 = 14$
 $n = 13 + 15 = 28$

$b(1) = B(1) + 1$
 $a(y) = a(y) + b(x) + 3$
 $a(1) = a(1) + b(2) + 3$
 $a(1) = 6 + 4 + 3 = 13$
 $a(x) = a(x) + 2$
 $a(1) = 5 + 2 = 7$
- RETURN 8

Resolucion por cadena dinamica.

CADON DYNAMIA.

① MAIN - PR

LE
LD

X = 1
Y = 1
Z = 1...6
a[1] = 6
a[2] = 5
a[3] = 4
a[4] = 3
a[5] = 2
a[6] = 1
b[1] = 2
b[2] = 3
b[3] = 4
b[4] = 5
b[5] = 6
b[6] = 7
b[7] = 8
b[8] = 10
b[9] = 12
b[10] = 14

Procédure D

FUNCTION F

VR

② Procédure D - PR

LE
LD

X =

Y =

H = 28

VR 8

③ FUNCTION F - PR

LE
LD

Y = 4

VR

Proc 1 = 1 to 6

2-1 a[1] = 6 b[1] = 2
2-2 a[2] = 5 b[2] = 4
2-3 a[3] = 4 b[3] = 6
2-4 a[4] = 3 b[4] = 8
2-5 a[5] = 2 b[5] = 10
2-6 a[6] = 1 b[6] = 12

D(a[2], b[abs(6-x+y)])

Intepre (13, 3)
(7, 4)
(4, 6)
(3, 8)
(2, 10)
(1, 14)

$$6 - x + y = 7$$

$$6 - 13 + 1 = -6$$

$$X = F + 5 \quad \left\{ \begin{array}{l} y = 8 + 2 \\ h = x + 15 = 28 \end{array} \right.$$

$$x = 8 + 5 = 13 \quad \left\{ \begin{array}{l} b(\text{abs}(6-x+y)) = b(\text{abs}(6-x+y)) + 2 = \\ b(6) = b(6) + 2 = \\ b(6) = 12 + 2 = 14 \end{array} \right.$$

$$b(x) = b(x) + 1 \quad \left\{ \begin{array}{l} x = x + 1 \\ a(y) = a(y) + b(x) + 3 \end{array} \right.$$

$$b(1) = b(1) + 1 \quad \left\{ \begin{array}{l} a(6) = a(6) + 1 = 2 \\ a(1) = a(1) + b(2) + 3 \end{array} \right.$$

$$b(1) = 2 + 1 = 3 \quad \left\{ \begin{array}{l} a(1) = 6 + 4 + 3 = 13 \end{array} \right.$$

$$a(x) = a(x) + 2 \quad \left\{ \begin{array}{l} y = y + 3 \\ \text{RETURN } b(y) \end{array} \right.$$

$$a(2) = a(2) + 2 = 4 \quad \left\{ \begin{array}{l} y = 1 + 3 = 4 \\ \text{RETURN } b(4) \end{array} \right.$$

$$a(2) = 5 + 2 = 7 \quad \left\{ \begin{array}{l} \text{RETURN } b(8) \end{array} \right.$$

Ejercicio 2

a) (10pts) Clasifique las siguientes estructuras de datos de acuerdo a lo visto en la práctica. Justifique en cada caso:

i) Java

```
class Alumno {  
    String nombre;  
    String apellido;  
    int edad;  
    float promedio;  
    String domicilio
```

```
    public float getPromedio(){  
        return this.promedio;  
    }  
}
```

ii) C

```
typedef struct _nodoArbol {  
    void *info;  
    struct _nodoArbol *hijoIzq;  
    struct _nodoArbol *hijoDer;  
} nodoArbol;  
  
typedef struct _arbolBinario {  
    int valor_guardado;  
    nodoArbol *raiz;  
} arbolBinario;
```

b) (10 pts) Responda si las siguientes afirmaciones son V o F. Justifique en cada caso

i) Los lenguajes con sistema de tipos fuerte son siempre compilados

ii) Las tuplas de python son un ejemplo de producto cartesiano

iii) La unión y la unión discriminada no son seguras en ejecución

Inciso a:

class Alumno: Producto cartesiano

- Justificación: es de tipo producto cartesiano ya que es el producto de varios tipos de datos de n tuplas ordenadas, donde cada componente pertenece a un conjunto diferente.

public float getPromedio(): Correspondencia finita:

- Justificación: es de tipo correspondencia finita ya que devuelve un unico valor.

typedef struct nodoArbol ()... : Producto cartesiano y recursion.

- Justificación: es de tipo producto cartesiano y recursion, porque es el producto de varios tipos de datos de n tuplas ordenadas, y recursion porque trabaja con instancia de si mismo que es donde hace referencia a su hijo izquierdo y hijo derecho.

typedef struct arbolBinario (): Producto cartesiano y recursion

- Justificación: es de tipo producto cartesiano ya que es el producto de varios tipos de datos de n tuplas ordenadas, y cada componente pertenece a un conjunto diferente. Además es de tipo recursion por que tiene un campo puntero a nodoArbol el cual es un tipo recursivo ya que permite elementos / instancias del mismo tipo.

Inciso b:

- Los lenguajes con sistema de tipos fuerte son siempre compilados:
Falso: No siempre tener tipado fuerte indica que son compilados. Un contraejemplo con este enunciado es que python es tipado fuerte y no es compilado, sino que es interpretado.
- Las tuplas de Python son un ejemplo de producto cartesiano:
Falso: Las tuplas en Python son un ejemplo de correspondencia finita, ya que tenemos un conjunto finito de valores y cada combinacion esta bien definida.
- La union y la union discriminada no son seguras en ejecucion:
Falso: La union no es segura porque no incluye informacion sobre el tipo seleccionado, lo que puede causar errores al acceder. La union discriminada si es segura, ya que incluye una etiqueta que permite saber que tipo esta presente en cada momento.

Ejercicio 4
(25pts). Marcar si son verdaderas o falsas las siguientes afirmaciones. Acompañar la respuesta con una justificación, caso contrario, NO se tomarán como válidas

a.	La sentencia for de ADA y Pascal son igualmente seguras	☒ V	☐ F
b.	El if de circuito corto puede prevenir errores en ejecución	☐ V	☒ F
c.	La sentencia yield de python equivale a hacer return	☐ V	☒ F
d.	En PL/1 si se genera una excepción, se ejecuta el manejador correspondiente y el control es pasado inmediatamente al programa principal	☐ V	☒ F
e.	La sentencia else de python en el manejo de excepciones se ejecuta solo si no se encontró ningún manejador asociado a la excepción en cuestión	☒ V	☐ F

a: FALSO: Pascal permite modificar el valor del indice dentro de la sentencia, mientras que ADA impone una restriccion que el indice NO puede modificarse dentro de esa iteracion.

b: VERDADERO: Si tenemos un if con dos condiciones con circuito corto, si la primer condicion a evaluar es falsa, la segunda condicion NO se evalua. Lo cual puede evitar errores en ejecucion como una posible division por cero, por ejemplo en este caso teniendo en cuenta que tenemos circuito largo, ocurriria una excepcion.

```
int num2=0;
int num1=5;
if (num1>10 && (num1 / num2) > 3)
```

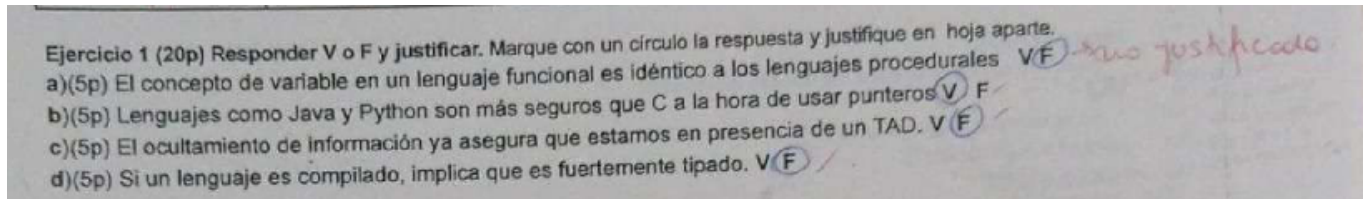
Por circuito corto como la primer condicion es falsa, no evalua la siguiente.

c: FALSO: return termina la funcion y devuelve un valor, mientras que el yield lo que hace es avanzar a la siguiente iteracion por ejemplo en un bucle for o while.

d: FALSO: En PL1, al tratar las excepciones por reasuncion, cuando se lanza una excepcion y se ejecuta el manejador, luego el control vuelve al punto donde ocurrio la excepcion.

e: FALSA: la sentencia else dentro de un try/except/else, se ejecuta SOLAMENTE si **NO** ocurrio ninguna excepcion.

Parcial 2024 segunda fecha:



a: FALSO :

- El concepto de las variables es distinto en cuanto a su mutabilidad. Ya que los lenguajes procedurales trabajan con variables mutables, es decir, pueden cambiar de valor. Mientras que los lenguajes funcionales trabajan con variables inmutables, es decir no puede cambiar de valor.

b: VERDADERO:

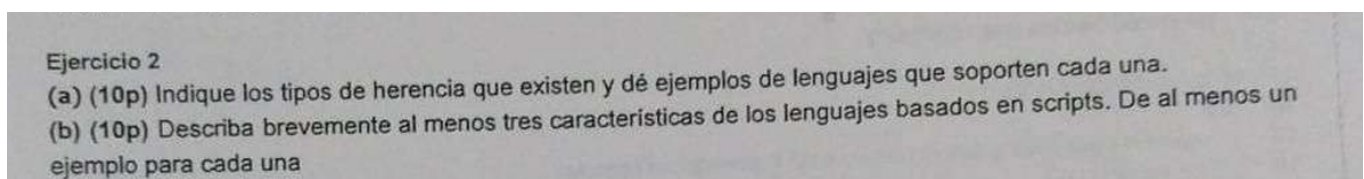
- Java y Python son mas seguros que C con respecto a la forma de pasar parametros, ya que C permite pasar punteros por parametro, esto puede producir mas errores a nivel de programa.

c: FALSO :

- El ocultamiento de informacion es una condicion necesaria, pero ademas, el lenguaje debe proveer encapsulamiento. Entonces, un TAD es posible gracias a las dos condiciones, por lo tanto si se provee una sola condicion no es suficiente para considerar un TAD.

d: FALSO :

- Un lenguaje compilado es aquel que se traducea codigo maquina antes de ejecutarse, mientras que un lenguaje fuertemente tipado es aquel que no permite operaciones entre tipos incompatibles, por ende estos conceptos no estan relacionados. Ademas, Python es un lenguaje interpretado y sin embargo es fuertemente tipado.



a:

- Herencia simple: Una clase puede heredar de una única clase base. Es el tipo de herencia más común y soportado por la mayoría de los lenguajes orientados a objetos. Por ejemplo Java, C#, Python soportan herencia simple
 - Herencia múltiple: Una clase puede heredar de múltiples clases bases, combinando sus atributos y comportamientos. Ejemplos C++, Python, Perl.
- b:
- Interpretados: Los lenguajes basados en script suelen ser interpretados en tiempo de ejecución, lo que significa que el código fuente se ejecuta línea por línea por un intérprete.
- Tipado dinámico: Muchos lenguajes de script son de tipado dinámico, lo que significa que el tipo de la variable puede cambiar en tiempo de ejecución.
- Los lenguajes de scripting asumen la existencia de componentes útiles en otros lenguajes. Su intención no es escribir aplicaciones desde el comienzo, sino por combinación de componentes.

Ejercicio 3 (30p) Sea el siguiente código escrito en Pascal like

a.

<pre> 1. 2. Procedure main 3. a: array(1..10) of integer; 4. x: integer; 5. i: integer; 6. Procedure Parcial (tipo_pasaje h: integer, tipo_pasaje m: integer) 7. Begin 8. x:=1; m:=m+1; 9. h:=m+x 10. x:=x+1; x=2 11. m:=m+x+a(3); 4+2+4=10 12. x:=x*2; a(5):=a(3)-1; x=4 13. write (m,a(5)); 14. m:=m+1; 15. End; 16. Begin 17. For i:=1 to 10 a(i):=i; 1 2 3 4 5 6 + 8 9 10 18. x:=3; i:=2; 19. Parcial(x, a(x+i)); 20. For i:=1 to 10 write (a(i)); 21. End. </pre>	a) (5p) Indique que tipo de pasaje de parámetros se debería usar si queremos que en la impresión realizada en la línea 13, a.1- ambas variables impriman valores distintos. a.2- ambas variables impriman valores iguales. b) (10p) Plantee diferencias, relacionada con la forma de implementación de cada uno y los resultados sobre este ejemplo considerando los siguientes tipos de pasajes parámetros: referencia y por valor. Puede realizar la pila y hacer comentarios en cada caso c) (5p) Si se utiliza pasaje por nombre para ambos parámetros ¿podría asegurarse que da el mismo resultado que alguno de los pasajes anteriores? Justifique la respuesta. d) (10p) ¿Qué sucedería si en el procedimiento Parcial se agrega la siguiente declaración: i: integer? Indique el resultado para cada uno de los tipos de pasaje de parámetros por nombre y por valor
---	--